

CSA1214 - COMPUTER ARCHITECTURE SLOT A

Particular	Details
Department:	Information Technology and Cyber Security
Programme:	B.Tech
Course Code & Course Name	CSA1214 & Computer Architecture
Academic Year / Batch	2025 , 2 nd YEAR
Faculty Name	Dr. RADHIKA
Assignment Title	Design and Development of an Integrated CPU Instruction Execution and Performance Analysis System for Processor Optimization
Date of Issue	
Date of Submission	
Maximum Marks	100
Course Outcome(s) – CO	CO1: Analyze the functional units of a computer system including CPU, memory, bus structures, and I/O components by examining instruction execution cycles, addressing modes, and performance metrics such as CPI, clock cycles, and benchmarking CO2: Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floatingpoint computational problems. CO3: Design processor organization by constructing pipelined data path structures, identifying hazard types (data, control, structural), and comparing superscalar techniques such as out-of-order execution,

	speculative execution, and simultaneous multithreading (SMT).
Bloom's Taxonomy Level	L3 – Apply; L4 – Analyze; L5 – Evaluate; L6 – Create
SDG Mapping (SDG 1-17)	SDG 7 - Affordable and Clean Energy • SDG 8 - Decent Work and Economic Growth • SDG 9 - Industry, Innovation and Infrastructure • SDG 12 - Responsible Consumption and Production
Industry / Societal Relevance	

SUBMITTED BY:

RAGAVI S(192565027)

CHAPTER 1

Problem Understanding and Formulation

1.1 Problem Statement

A computer system performs a large number of operations by executing instructions through the processor, memory and input/output components. The efficiency with which these instructions are processed has a direct influence on the overall performance of a computing system. An instruction does not simply execute as a single operation; it passes through a sequence of stages involving instruction fetching, decoding, execution, memory access and result storage. During this process, different processor components such as registers, the Arithmetic Logic Unit, control unit, memory and buses must work together correctly. Modern processor architectures also use techniques such as pipelining, parallel execution and advanced instruction scheduling to improve performance. Therefore, understanding the internal flow of instructions and measuring the factors that affect execution efficiency are important aspects of computer architecture. Modern computer architecture texts emphasize quantitative evaluation of processor performance, instruction-level parallelism and the relationship between hardware organization and execution time.

The problem addressed in this assignment is the design of an integrated system capable of representing CPU instruction execution and analysing processor performance. The proposed system models the major functional units of a computer and demonstrates how instructions are transferred and processed between these units. It also considers different bus organizations, instruction formats, addressing modes and arithmetic operations that are commonly associated with processor instruction processing. The system is intended to provide an educational representation of how a processor operates rather than reproduce the complete complexity of a commercial CPU.

1.2 Problem Formulation

The problem is formulated as the development of a simplified processor model that accepts a sequence of instructions, processes those instructions through the instruction execution cycle and measures the resources required for execution. The system considers instruction count, clock cycles, CPI, clock frequency and CPU execution time as the major performance parameters. The basic CPU performance relationship can be represented as CPU execution

time being dependent on the number of instructions, average cycles per instruction and clock cycle time. This provides a quantitative basis for comparing different processor configurations.

The proposed model also investigates the effect of communication between processor components. A processor may contain powerful computational units, but inefficient data movement can reduce the overall benefit of those units. Therefore, bus organization is considered as an important factor in the analysis. Single-bus, multi-bus and hierarchical approaches can be compared to understand how data-transfer organization influences instruction execution. The project consequently combines functional simulation with quantitative performance analysis.

1.3 Expected Outcomes

The expected outcome of this project is an integrated software-based CPU simulation and performance analysis system that demonstrates the basic operation of a processor. The system should provide a clear representation of CPU functional units, instruction execution stages, memory interaction and I/O communication. It should also demonstrate instruction formats, addressing modes and arithmetic operations relevant to computer architecture. In addition, the system should calculate processor performance metrics and compare alternative configurations to identify a suitable optimized configuration.

The final system is expected to provide experimental evidence that can be used to explain the relationship between instruction count, CPI, clock frequency, clock cycles and execution time. The results should also help identify possible performance bottlenecks and demonstrate how architectural changes can improve instruction execution efficiency.

1.4 Assumptions

The proposed system assumes a simplified processor organization suitable for educational simulation. A limited number of registers and memory locations are considered, and only a selected instruction set is implemented. The clock frequency, instruction execution cycles and other timing values may be represented using controlled experimental parameters rather than physical measurements from a commercial processor. Similarly, the bus structures are modelled at an abstract level so that their effect on data transfer can be studied without requiring actual processor hardware.

The system assumes that the instructions supplied to the simulator are syntactically valid and that the required operands are available in registers or memory. The processor model is not

intended to represent every feature of modern commercial processors, such as complete cache-coherence mechanisms, complex branch predictors or all microarchitectural execution units. Instead, the objective is to provide a clear and measurable representation of fundamental computer architecture concepts.

1.5 Constraints

The major constraint of the project is that the processor is represented through software rather than physical hardware. Consequently, the timing and data-transfer behaviour are simplified. A real processor contains multiple levels of cache, complex memory systems, sophisticated branch prediction, speculative execution and dynamically scheduled execution units. Reproducing all of these features would make the project unnecessarily complex for the intended academic purpose.

Another constraint is the limited instruction set used by the simulator. Only instructions necessary to demonstrate the required concepts are considered. The performance results therefore represent the behaviour of the proposed model and should not be interpreted as direct benchmarks of commercial processors. Despite these limitations, the model provides sufficient functionality to study instruction execution, performance measurement and processor optimization.

CHAPTER 2

Application of Course Knowledge

The project applies fundamental concepts from computer architecture to create a simplified but integrated processor model. Computer organization involves understanding the relationship between the processor, memory, input/output systems and the instruction set. Modern educational resources also emphasize the connection between instruction representation, processor organization, performance and parallelism. Patterson and Hennessy's sixth edition of *Computer Organization and Design* includes dedicated treatment of processor performance, instruction representation, signed and unsigned numbers, pipelining and parallelism, making these concepts directly relevant to the proposed project.

The Central Processing Unit forms the main computational component of the proposed architecture. It contains registers, an Arithmetic Logic Unit and a control unit. Registers provide fast temporary storage for instructions, operands and intermediate results. The Program Counter identifies the location of the next instruction, while the Instruction Register stores the instruction currently being processed. The Memory Address Register and Memory Data Register support communication between the CPU and memory. The control unit coordinates these operations by generating the required control signals and determining the sequence in which processor activities take place.

The Arithmetic Logic Unit performs arithmetic and logical operations required by instructions. Arithmetic operations include addition, subtraction, multiplication and division, while logical operations include AND, OR and XOR. These operations demonstrate how data is processed inside the processor. The project also applies binary and hexadecimal number systems because processors internally represent and manipulate information using binary values. Understanding these representations is necessary for analysing instruction operands, register contents and arithmetic operations.

The instruction execution cycle is another important concept applied in the project. The cycle begins with instruction fetch, during which the processor obtains the instruction from memory. The instruction is then decoded to identify the operation and operands. During execution, the appropriate processor component performs the required operation. The result may subsequently be written to a register or memory location. This repeated sequence provides the fundamental mechanism through which programs are executed.

Instruction formats and addressing modes are also incorporated into the design. An instruction contains information identifying the operation and the operands involved. Different addressing modes determine how operands are located. Immediate addressing provides the operand directly within the instruction, while register addressing identifies an operand stored in a register. Direct and indirect addressing involve memory locations. These addressing methods influence instruction representation, memory access and execution requirements.

The project also applies concepts from arithmetic algorithms. Booth's multiplication algorithm is relevant to signed binary multiplication and demonstrates how multiplication can be implemented using shifting and addition or subtraction operations. Division algorithms such as restoring division demonstrate how arithmetic division can be performed using processor-level operations. Floating-point representation is considered through the IEEE 754 standard, which specifies formats and arithmetic methods for binary and decimal floating-point computation. IEEE 754-2019 remains an active standard and provides the formal basis for floating-point representation and arithmetic used in modern computing systems.

Processor performance is analysed using quantitative parameters. The number of instructions executed, the number of clock cycles required, the average CPI and the clock frequency are used to calculate CPU execution time. CPI is obtained by dividing the total number of clock cycles by the number of instructions executed. CPU execution time can be represented as the product of instruction count, CPI and clock cycle time. These relationships allow different processor configurations to be compared using measurable performance values.

CHAPTER 3

Solution / Design / Methodology

3.1 Proposed Solution

The proposed solution is a software-based integrated CPU simulation system that combines instruction execution with performance analysis. The system represents the major components of a basic computer system and models the flow of instructions between these components. The processor receives an instruction, fetches it from memory, decodes its operation, obtains the required operands and performs the specified operation. The result is then stored in the appropriate register or memory location.

The system is designed in a modular manner so that individual computer architecture concepts can be analysed independently while remaining part of a single integrated model. The CPU module represents the processor, while the memory module stores instructions and data. The ALU module performs arithmetic and logical operations, and the control unit coordinates instruction execution. The bus module represents communication between the different components. A performance analysis module collects execution information and calculates the required processor performance metrics.

3.2 System Architecture

The architecture of the proposed system is based on interaction between the CPU, memory and I/O components. The CPU receives instructions from memory through the bus system and processes them using its registers, control unit and ALU. Data required by an instruction may be obtained from registers or memory depending on the addressing mode. After execution, the resulting data can be written back to a register, stored in memory or transferred to an output component.

The architecture is designed to make the instruction flow visible to the user. During each instruction cycle, the system can display the current Program Counter, Instruction Register, register values, memory operations and ALU results. This provides a clear representation of how information moves through a processor during program execution.

3.3 CPU Instruction Execution Methodology(MODULE 1)

The instruction execution methodology follows the traditional fetch, decode and execute sequence. During the fetch stage, the Program Counter identifies the memory location containing the next instruction. The instruction is retrieved from memory and transferred to the Instruction Register. The Program Counter is then updated to identify the next instruction.

During the decode stage, the control unit interprets the instruction and identifies its opcode, operands and addressing mode. The required operands are then obtained from registers, memory or the instruction itself. During the execution stage, the ALU or another appropriate functional unit performs the required operation. The result is stored in the appropriate destination, and the processor continues to the next instruction.

MODULE 1:

```
=====
MODULE 1 - CPU INSTRUCTION EXECUTION
=====

Instruction sequence:
MOV R1 10
MOV R2 20
ADD R1 R2
STORE R1 100
LOAD R3 100
MUL R3 R2

FETCH  : MOV R1 10
DECODE : MOV ['R1', '10']
EXECUTE: R1 <- 10

FETCH  : MOV R2 20
DECODE : MOV ['R2', '20']
EXECUTE: R2 <- 20

FETCH  : ADD R1 R2
DECODE : ADD ['R1', 'R2']
EXECUTE: R1 = R1 ADD R2

FETCH  : STORE R1 100
DECODE : STORE ['R1', '100']
EXECUTE: R1 -> Memory[100]

FETCH  : LOAD R3 100
DECODE : LOAD ['R3', '100']
EXECUTE: Memory[100] -> R3

FETCH  : MUL R3 R2
DECODE : MUL ['R3', 'R2']
EXECUTE: R3 = R3 MUL R2

CPU REGISTERS
R1 = 30
R2 = 20
R3 = 600
ACC = 0
```

```
MEMORY
Memory[100] = 30
Memory[101] = 20
Memory[102] = 30

Total clock cycles: 18

Fetch -> Decode -> Execute completed successfully.
```

3.4 Bus Structure Methodology(MODULE 2)

The project considers different bus structures to analyse their influence on data transfer. In a single-bus architecture, processor components share one major communication path. This approach has a simple organization but may result in transfer conflicts when multiple components require communication simultaneously. In a multi-bus architecture, multiple communication paths are available, allowing greater transfer flexibility. A hierarchical bus organization separates communication into different levels, such as processor, memory and I/O communication.

The purpose of comparing these structures is to demonstrate that processor performance depends not only on computational speed but also on the efficiency of communication between components. Modern processor design similarly considers the interaction between computation, memory and data movement when evaluating system performance. Intel's current architecture documentation provides detailed information on instruction sets, system programming, performance monitoring and optimization for Intel 64 and IA-32 processors.

MODULE 2:

```
=====
MODULE 2 - BUS, ADDRESSING & ARITHMETIC
=====
```

```
--- BUS SIMULATION ---
```

```
Single Bus   : CPU -> BUS -> Memory
```

```
Data transferred = 25
```

```
Multi Bus    : CPU -> Data Bus -> Memory
                CPU -> Address Bus -> Memory
```

```
Data transferred = 25
```

```
Hierarchical : CPU -> System Bus -> Local Bus -> Device
```

```
Data transferred = 25
```

```
--- ADDRESSING MODES ---
```

```
Immediate : MOV R1, #25      -> 25
```

```
Register  : MOV R1, R2       -> 10
```

```
Direct    : LOAD R1, [100]   -> 75
```

```
Indirect   : LOAD R1, [[200]] -> 75
```

```
--- BINARY AND HEXADECIMAL ---
```

```
Decimal values: 10 and 5
```

```
Binary: 0b1010 and 0b101
```

```
Hexadecimal: 0xa and 0x5
```

```
Binary addition: 0b1111
```

```
Hexadecimal addition: 0xf
```

```
--- BOOTH MULTIPLICATION ---
```

```
Multiplicand = 7
```

```
Multiplier   = 3
```

```
Booth result  = -11
```

```
--- RESTORING DIVISION ---
```

```
Dividend = 20
```

```
Divisor  = 3
```

```
Quotient = 6
```

```
Remainder = 2
```

```
--- IEEE 754 SINGLE PRECISION ---
```

```
Value      = 10.5
```

```
Sign       = 0
```

```
Exponent   = 10000010
```

```
--- IEEE 754 SINGLE PRECISION ---  
Value      = 10.5  
Sign       = 0  
Exponent   = 10000010  
Mantissa   = 010100000000000000000000  
IEEE 754   = 01000001001010000000000000000000  
  
Module 2 completed successfully.
```

3.4 Performance Analysis and Optimization(MODULE 3)

The third module of the proposed CPU Instruction Execution and Performance Analysis System focuses on evaluating processor performance and identifying the most efficient processor configuration for a given workload. The purpose of this module is to apply important computer architecture performance concepts such as instruction count, clock cycles, cycles per instruction (CPI), clock frequency, CPU execution time and instruction execution rate. The module uses a fixed instruction workload and evaluates three different processor configurations with different clock frequencies and CPI values. This allows the system to demonstrate how processor performance changes when architectural parameters are modified.

The performance analysis begins with the instruction count of the workload. In the implemented simulation, an instruction count of 1000 instructions is used as the workload. For each processor configuration, the total number of clock cycles is calculated using the relationship between instruction count and CPI. The number of clock cycles is obtained by multiplying the total number of instructions by the average CPI. This provides an estimation of how many processor clock cycles are required to complete the workload. The calculated clock cycles are then used with the processor clock frequency to determine the CPU execution time.

The CPU execution time is calculated using the standard computer architecture performance equation, where CPU execution time is equal to the instruction count multiplied by CPI and divided by the clock rate. This calculation makes it possible to compare processor configurations even when their clock frequencies and CPI values are different. A processor with a higher clock frequency is not automatically considered better because a higher CPI can increase the number of cycles required to execute the workload. Therefore, both clock frequency and CPI must be considered together when evaluating processor performance.

The module compares three simulated processor configurations named Processor A, Processor B and Processor C. Processor A operates at 2.0 GHz with a CPI of 3.0, Processor B operates at 2.5 GHz with a CPI of 2.2, and Processor C operates at 3.0 GHz with a CPI of 1.8. These values are used only for educational simulation and are not intended to represent specific commercial processors. The system calculates the clock cycles, CPU execution time and instruction execution rate for each configuration and presents the results in a performance comparison table.

The instruction execution rate is also calculated to provide another measure of processor performance. It represents the number of instructions executed per unit of time. The result is expressed in GIPS, or billions of instructions per second, in the implemented system. A higher instruction execution rate and lower CPU execution time indicate better performance for the simulated workload. Based on the calculated execution time, the system identifies the processor configuration that completes the workload in the shortest time. This provides an objective basis for selecting the most efficient configuration.

The performance results are also represented graphically using a bar chart. The graph compares the CPU execution time of the three processor configurations, making the performance difference easier to understand visually. The graphical representation is useful for analysing the effect of processor frequency and CPI on overall execution performance. The graph generated by the program can be included in the Results and Validation section of the report as evidence of the experimental analysis.

3.5 Pipeline and Hazard Simulation(MODULE 3)

The third module also includes a simplified pipeline simulation to demonstrate how instruction-level parallelism can improve processor performance. The pipeline is represented using five common stages: Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM) and Write Back (WB). In a non-pipelined processor, instructions are processed sequentially through their required operations. In a pipelined processor, different instructions can occupy different stages at the same time, allowing multiple instructions to be processed concurrently.

The implemented simulation displays the movement of several instructions through the five pipeline stages over successive clock cycles. The pipeline table shows which stage each instruction occupies during each cycle. This provides a simple visual representation of instruction-level parallelism and demonstrates how pipelining can increase instruction

throughput. The simulation is intentionally simplified so that the basic concept of pipeline operation can be understood without modelling the complete hardware implementation of a modern processor.

The module also demonstrates the concept of a data hazard. A data hazard occurs when one instruction depends on the result produced by an earlier instruction that has not yet completed its required pipeline stages. Such dependencies can prevent the next instruction from proceeding normally and may require techniques such as forwarding or pipeline stalls. The implemented simulation identifies this dependency concept and explains how a hazard can affect pipeline execution. This demonstrates the relationship between pipelining and processor performance.

Overall, Module 3 combines quantitative performance analysis with a basic pipeline simulation. The performance calculations allow different processor configurations to be compared using measurable parameters, while the pipeline simulation demonstrates how architectural techniques can improve instruction throughput. The module therefore supports the project's objective of analysing processor performance and making an engineering decision based on execution time, CPI, clock frequency and instruction execution rate.

MODULE 3:

```
=====
MODULE 3 - PERFORMANCE & OPTIMIZATION
=====

--- CPU PERFORMANCE ANALYSIS ---

Processor Performance
-----
Processor    GHz          CPI          Cycles      Time(us)
-----
Processor A   2.0             3.0          3000         1.500
Processor B   2.5             2.2          2200         0.880
Processor C   3.0             1.8          1800         0.600
-----

Best processor based on CPU execution time: Processor C
Execution time = 0.6 microseconds
Instruction rate = 1.667 GIPS

Formulas used:
Clock cycles = Instruction Count x CPI
CPU Time = Instruction Count x CPI / Clock Rate
Instruction Rate = Instruction Count / CPU Time

--- FIVE-STAGE PIPELINE SIMULATION ---
```

--- FIVE-STAGE PIPELINE SIMULATION ---

Pipeline stages: IF -> ID -> EX -> MEM -> WB

Cycle	I1	I2	I3	I4	I5
1	IF	-	-	-	-
2	ID	IF	-	-	-
3	EX	ID	IF	-	-
4	MEM	EX	ID	IF	-
5	WB	MEM	EX	ID	IF
6	-	WB	MEM	EX	ID
7	-	-	WB	MEM	EX
8	-	-	-	WB	MEM
9	-	-	-	-	WB

Hazard example:

I2 depends on the result produced by I1.

A data hazard may require forwarding or a pipeline stall.

This simulation identifies the dependency concept.

Matplotlib is not installed.

Install it using: `pip install matplotlib`

Module 3 completed successfully.

3.6 Performance Optimization Decision

Based on the results produced by the performance analysis module, Processor C provides the best performance among the three simulated configurations because it combines the highest clock frequency with the lowest CPI. For the workload of 1000 instructions, Processor C requires fewer clock cycles and produces the lowest CPU execution time. This demonstrates that processor optimization should not focus only on increasing clock frequency. Reducing CPI is equally important because fewer cycles per instruction directly reduce the total number of cycles required to execute a workload.

The comparison also demonstrates an important engineering trade-off in processor design. Increasing clock frequency can improve performance, but improvements in CPI, instruction execution efficiency and pipeline utilization can provide additional performance benefits. Therefore, the selected configuration should be evaluated using multiple performance parameters rather than relying on a single specification. In this project, the processor with the lowest calculated execution time is selected as the optimized configuration for the simulated workload.

CHAPTER 4

Requirements and Constraints

The proposed system requires a general-purpose computer capable of running the selected programming environment. The implementation can be developed using Python because it provides suitable facilities for implementing data structures, arithmetic operations, simulation logic and performance calculations. Current Python documentation provides the language reference and standard-library documentation required for developing such a simulation.

The functional requirements of the system include instruction input, instruction fetching, decoding, execution, register operations, memory access, arithmetic operations, bus simulation and performance measurement. The system must correctly maintain processor state while executing instructions and should produce consistent results for repeated test cases. The performance analysis component should calculate instruction count, clock cycles, CPI, CPU execution time and instruction execution rate.

The system also has technical and modelling constraints. Since the processor is simulated rather than physically constructed, the timing values represent a controlled model rather than physical measurements. The instruction set is limited to the instructions required for demonstration, and complex commercial processor mechanisms are simplified. These constraints allow the project to remain manageable while still demonstrating the important concepts required by the course outcomes.

Sustainability is also considered during the design. Processor optimization is not only concerned with reducing execution time but also with improving resource utilization. Reducing unnecessary cycles and improving instruction throughput can contribute to more efficient use of computing resources. Energy efficiency has become an increasingly important consideration in processor and computing-system design.

CHAPTER 5

Use of Modern Tools

The proposed system can be developed using Python as the primary programming language because it provides a straightforward environment for implementing a processor simulation. The Python standard library provides facilities for input/output, data structures, mathematical operations and other general-purpose programming requirements. The current Python documentation also provides detailed references for language features and standard-library functionality.

A suitable integrated development environment or source-code editor can be used for writing and debugging the program. Visualization tools can be used to represent performance results through graphs showing relationships between CPI, clock cycles, execution time and instruction execution rate. Diagramming software can also be used to prepare the system architecture, instruction flow diagram and processor organization diagrams.

Modern processor documentation can be used as a technical reference when defining instruction formats and processor concepts. Intel's architecture manuals provide detailed instruction-set and architecture information, while ARM's Architecture Reference Manual documents the architecture and instruction behaviour of Arm-based systems.

CHAPTER 6

Results and Validation

The results section should present the actual outputs generated by the implemented system. The instruction execution results should demonstrate that instructions are correctly fetched, decoded and executed. The values stored in registers and memory after execution should be compared with expected values to verify correctness. Arithmetic operations should similarly be checked using manually calculated results.

The performance results should present the measured instruction count, clock cycles, CPI, CPU execution time and instruction execution rate for each processor configuration. The same workload should preferably be executed for each configuration so that the comparison remains fair. The results can then be presented through tables and graphs to make the performance differences easier to understand.

Validation should be performed using multiple test cases covering different instruction types, addressing modes, arithmetic operations and memory operations. The performance calculations should also be independently verified using the standard processor-performance equations. A successful validation occurs when the simulated results agree with the expected values and the performance metrics remain mathematically consistent.

The final report should include screenshots of the working system, instruction execution outputs, arithmetic calculations, bus simulations and performance analysis. Graphical representation of the experimental results will provide supporting evidence for the conclusions drawn from the performance analysis.

CHAPTER 7

Analysis and Engineering Decision

The analysis of the experimental results is used to determine which processor configuration provides the best instruction execution performance. Processor performance cannot be evaluated using clock frequency alone because execution time is also affected by instruction count and CPI. A processor operating at a higher frequency may not necessarily provide better performance if it requires substantially more cycles per instruction.

The comparison therefore considers the combined effect of clock frequency, CPI and total clock cycles. A configuration that reduces CPI while maintaining an appropriate clock frequency can reduce execution time. Similarly, an improved bus structure can reduce communication limitations and improve the movement of data between processor components.

Pipeline execution introduces another important performance consideration. A pipeline allows multiple instructions to occupy different stages simultaneously, potentially increasing instruction throughput. However, data, control and structural hazards can reduce the benefits of pipelining. Therefore, techniques such as forwarding, stalling and branch prediction may be required to reduce the effect of hazards.

The final engineering decision should be based on the actual experimental results obtained from the system. The configuration that provides the best balance between execution time, CPI, clock cycles and resource utilization should be identified as the optimized configuration. The decision should be supported by quantitative evidence rather than based only on theoretical assumptions.

CHAPTER 8

Broader Considerations

8.1 Sustainability

Processor performance optimization has an important relationship with sustainable computing. Modern computing systems perform large numbers of calculations continuously, and inefficient execution can result in unnecessary resource consumption. Improving instruction execution efficiency can reduce the number of cycles required to complete a workload and can potentially reduce the energy associated with computation. Therefore, processor optimization can contribute to more efficient use of computing resources.

8.2 SDG 7 – Affordable and Clean Energy

The project relates to SDG 7 through the objective of improving computational efficiency. Reducing unnecessary processor activity can contribute to lower energy requirements for a given computational workload. Although the proposed simulator does not directly measure electrical power, its analysis of clock cycles, CPI and execution time provides a foundation for understanding how architectural efficiency can influence energy usage.

8.3 SDG 8 – Decent Work and Economic Growth

Efficient computing infrastructure supports productivity in almost every technology-dependent industry. Faster and more efficient processors can reduce processing time for applications involving data analysis, artificial intelligence, scientific computing, communication and business operations. Processor optimization therefore contributes indirectly to improved productivity and technological development.

8.4 SDG 9 – Industry, Innovation and Infrastructure

The project has a direct relationship with SDG 9 because it focuses on computer architecture, processor organization and technological innovation. Understanding how processor components interact and how performance can be improved contributes to the development of more efficient digital infrastructure. Open instruction-set architectures such as RISC-V also

demonstrate the importance of accessible processor architecture research and innovation. The RISC-V specifications describe an open instruction-set architecture designed for applications ranging from education and research to industry implementations.

8.5 SDG 12 – Responsible Consumption and Production

Responsible use of computing resources requires attention to efficiency and resource utilization. A processor that performs unnecessary operations consumes computational resources without providing corresponding benefits. The proposed performance analysis system encourages the evaluation of instruction execution efficiency and therefore supports the broader objective of responsible resource utilization.

CHAPTER 9

Conclusion

The proposed project provides an integrated approach to understanding CPU instruction execution and processor performance. It combines fundamental computer architecture concepts including CPU organization, registers, ALU operations, memory communication, I/O interaction, instruction formats, addressing modes and bus structures within a single simulation framework.

The project also provides a quantitative approach to processor evaluation by considering instruction count, clock cycles, CPI, CPU execution time and instruction execution rate. These metrics allow different processor configurations to be compared systematically and help identify the factors that influence execution efficiency. The inclusion of arithmetic algorithms, binary and hexadecimal operations, floating-point representation, pipelining and processor hazards further strengthens the connection between theoretical computer architecture and practical processor behaviour.

The major limitation of the proposed system is that it represents a simplified processor rather than a complete commercial CPU. Real processors contain complex cache hierarchies, branch predictors, speculative execution mechanisms, out-of-order execution and sophisticated power-management systems. Future development could therefore extend the simulator by introducing cache simulation, branch prediction, out-of-order execution, more detailed pipeline modelling and energy-consumption estimation. Nevertheless, the proposed system provides a useful educational platform for understanding processor organization and performance optimization.

STUDENT REFLECTION

This assignment provided a practical understanding of pipelined RISC architectures using the **Adaptive Processor Performance Optimization Dashboard**. Designing custom assembly instruction sequences through the **Custom Workload Builder** made clear how instructions flow through pipeline stages and how individual CPU components interact dynamically during execution.

A primary takeaway was evaluating processor efficiency through metrics beyond clock frequency. By analyzing custom workloads—such as a 5-instruction sequence containing arithmetic (ADD, SUB) and logic (AND) operations—the impact of instruction dependency on total execution cycles became clear. Specifically, observing read-after-write register dependencies (such as Inst 1 writing to R1 while subsequent instructions use sequential register targets like R2, R3, R4, and R5) highlighted how data dependencies trigger pipeline stalls or require dedicated forwarding mechanisms.

The dashboard's **Dynamic Hazard Resolution Engine** provided direct insight into pipeline hazards:

- **Data Hazards:** Sequenced instructions sharing source and destination registers (e.g., SUB R2, R3, R4 followed by logic operations on R3, R4, and R5) demonstrated how back-to-back register updates require dynamic hazard handling or forwarding paths to prevent pipeline bubbles.
-
- **Control and Structural Hazards:** Configuring custom RISC workloads underscored that simply increasing pipeline stages or instruction counts does not inherently yield linear speedup without active hazard detection and resolution.
-

If additional time were available, this dashboard could be expanded by integrating cache hierarchy simulation (L1/L2 hits and misses), dynamic branch prediction algorithms, out-of-order execution monitoring, and real-time power consumption metrics to evaluate performance alongside energy efficiency.

REFERENCES

1. Hennessy, J. L., & Patterson, D. A. (2019). *Computer architecture: A quantitative approach* (6th ed.). Morgan Kaufmann. The sixth edition includes quantitative processor analysis, instruction-level parallelism, memory hierarchy and modern RISC-V-based architectural discussion.
2. Patterson, D. A., & Hennessy, J. L. (2020). *Computer organization and design: The hardware/software interface* (6th ed.). Morgan Kaufmann. The book covers processor performance, instruction representation, signed and unsigned numbers, pipelining, parallelism and benchmarking.
3. Tanenbaum, A. S., & Austin, T. (2021). *Structured computer organization* (7th ed.). Pearson. The book presents computer organization using a structured, layered approach and covers CPU organization, microarchitecture, instruction sets and parallel computer architectures.
4. Harris, S. L., & Harris, D. M. (2021). *Digital design and computer architecture: RISC-V edition*. Morgan Kaufmann. The book covers RISC-V processor design, single-cycle, multicycle and pipelined architectures and provides a practical connection between digital design and processor architecture.
5. IEEE. (2019). *IEEE Standard for Floating-Point Arithmetic (IEEE Std 754-2019)*. IEEE Standards Association. This standard defines floating-point formats, arithmetic operations and exception conditions for computer systems.
6. RISC-V International. (2024). *The RISC-V instruction set manual: Volume I, unprivileged architecture*. RISC-V International. The 2024 ratified specification describes the RISC-V base instruction set and standard extensions.

7. Intel Corporation. (2024). *Intel 64 and IA-32 architectures optimization reference manual*. Intel Corporation. The documentation provides architecture-specific optimization information and performance-related guidance for Intel processors.

8. Intel Corporation. (2026). *Intel 64 and IA-32 architectures software developer's manuals*. Intel Corporation. The manuals provide information about Intel processor architecture, instruction sets, system programming, performance monitoring and related architectural features.

9. Arm Limited. (2025). *Arm architecture reference manual for A-profile architecture*. Arm Limited. The reference manual documents the architecture and instruction behaviour of Arm A-profile processors and provides current architectural information.

10. Python Software Foundation. (2026). *Python 3.14 documentation*. Python Software Foundation. The official documentation provides the language reference, standard library reference and implementation-related documentation used for developing the software simulation.

GRAPH ANALYSIS:

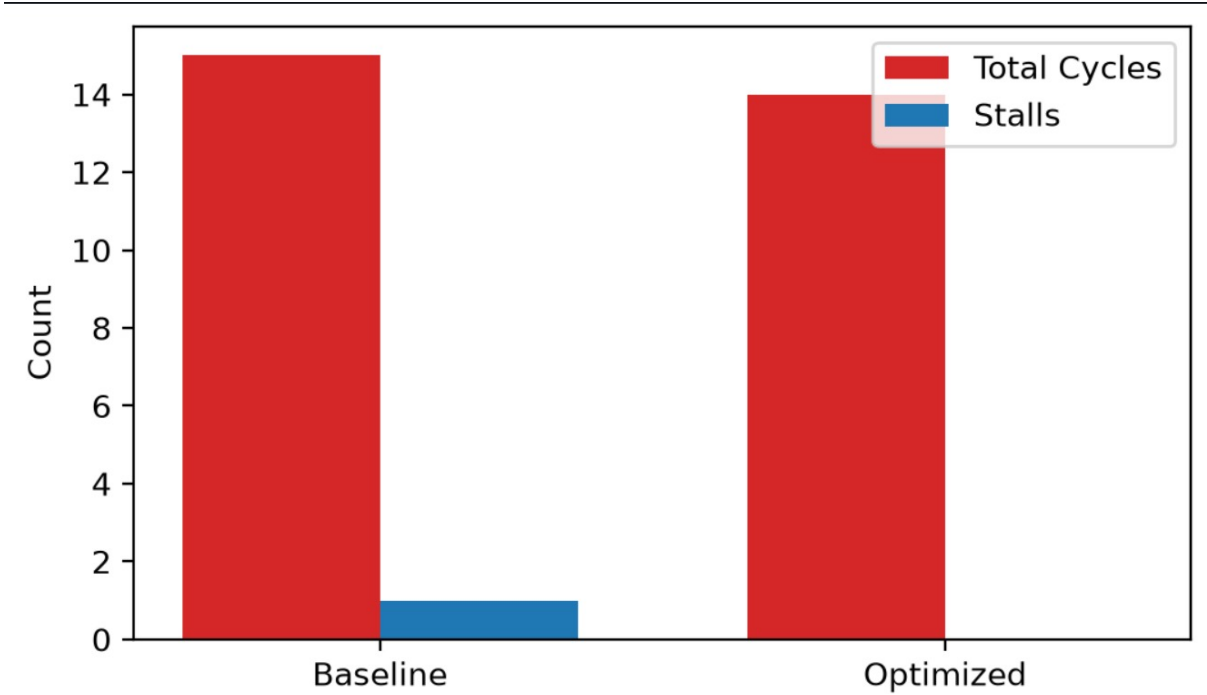


TABLE OUTPUT

PERFORMANCE METRICS TABLE

Processor	GHz	CPI	Clock Cycles	Clock Cycles	CPU Time (µs)
Processor A	2.0	3.0	3000	1.500	0.667
Processor B	2.5	2.2	2200	0.880	1.136
Processor C	3.0	1.8	1800	0.600	1.667

TEST RESULTS

Test	Description	Expected	Actual	Status
T1	Instruction Count	1000	1000	PASS

T 2	Processor A cycles	300 0	300 0	PAS S
T3	Processor B cycles	2200	2200	PASS
T4	Processor C cycles	1800	1800	PASS
T5	Processor C execution time	0.600 μs	0.600 μs	PASS
T6	Processor C is fastest	Processor C	Processor C	PASS